

Developer & Admin Guide — START HERE

Onboarding for anyone picking up the AMW Performance Evaluation System. Read this first, then dive into the focused docs linked at the bottom.

Golden rule — never commit real employee data. The repo holds code + a *synthetic* demo seed only. Real names, employee numbers, DOB, and salaries live in the database (runtime) and in CSV files kept **outside** the repo. See [Security & data rules](#).

1. What this is

A full-stack annual performance-evaluation platform:

- **Frontend** — React 18 + TypeScript + Vite (custom "KBT" / SAP-Fiori design system, Thai/EN/FR i18n).
- **Backend** — Express + Prisma 5 + PostgreSQL, JWT auth.
- **Tooling** — Vitest / Playwright / node:test, a custom QA Robot UI suite, Docker + GitHub Actions CI.

The product surface, architecture diagrams, and screenshots live in the [README](#).

2. Local setup

Prerequisites: Node 20+, npm, Docker (for PostgreSQL), Git.

```
git clone <repo> && cd Performance-evaluation-Form
npm install                    # installs all workspaces (root, frontend, backend)

# Database (Docker) + env
docker compose up -d postgres # local PostgreSQL on :5432
cp .env.example backend/.env   # then edit DATABASE_URL / JWT_SECRET if needed

# Migrate + seed demo data
npm run db:deploy              # apply migrations
npm run db:seed                # synthetic demo accounts + one demo evaluation

# Run both servers (backend :3001, frontend :5173)
npm run dev
```

Open <http://localhost:5173>. **Demo login IDs/passwords are not in the repo** — read them from [backend/prisma/seed.cjs](#). Quick-login chips on the login screen cover each role.

3. Project structure

```
frontend/  React app (features/, components/, lib/, i18n/)
backend/   Express API (src/{routes,controllers,services,middleware,config}, prisma/)
scripts/   qa-robot/ (UI test bot), deploy + maintenance scripts
docs/      this guide + focused references (api, data-model, runbook, ...)
docker-compose.*.yaml  dev / demo / prod / monitoring stacks
.github/workflows/      ci.yaml (build·scan·push), deploy.yaml, backup.yaml
```

Where common things live:

Looking for...	File
Evaluation form UI	<code>frontend/src/features/evaluations/EvaluationFormPage.tsx</code>
Form definitions (sections per level)	<code>frontend/src/features/evaluations/constants/formDefinitions.ts</code>
Competencies per position (CC/MC/TCM)	<code>frontend/src/features/evaluations/constants/competency.ts</code>
PDF export	<code>frontend/src/lib/download.ts</code>
Score scale / tiers	<code>frontend/src/lib/score.ts</code>
Auth store (token, session)	<code>frontend/src/features/auth/authStore.ts</code>
Translations (en/th/fr)	<code>frontend/src/i18n/translations.ts</code>
API env validation	<code>backend/src/config/env.ts</code>
Attendance / scoring logic	<code>backend/src/services/attendanceService.ts</code> , <code>evaluationSectionService.ts</code>
DB schema	<code>backend/prisma/schema.prisma</code>
Demo seed (synthetic)	<code>backend/prisma/seed.cjs</code>

4. Key concepts & conventions

- **Scoring is a single 1–5 scale** everywhere — form, PDF, dashboard, reports, calibration. (The old 0–4 "GPA" display was retired; `toGpa` in `score.ts` is now a pass-through.) Weighted total = Goal 60–70% + Competency/Evaluation 20% + Attendance 10% + Training 0–10%.
- **Auth** — JWT (HS256, 12h). The frontend persists the token in `sessionStorage` (`auth-storage` , via Zustand `persist`) so a **reload keeps the session**. If you touch `authStore.ts` , keep `token` in `partialize` or refresh silently logs users out.
- **RBAC** — `DEVELOPER` = everything; `ADMIN` = management + `calibrate` (blocked from editing evaluation content); `MANAGER` /supervisory = create cycles + evaluate; `EMPLOYEE` = own data. Backend enforces it (`requireRole` , `authorizeEvaluation`); the UI hides controls per role.
- **Forms** — two shapes: *weighted* (manager/appraisal) and *OSE* (officer level). The active shape comes from `formType` → `getFormDefinition` . Competency descriptions are **fixed per position** (the evaluator only scores).
- **PDF** — `jsPDF` with dual fonts: a Thai font (U+0E00–0E7F) + Helvetica Latin. Use `setTextFont(text)` (picks the font per string) — text drawn with the wrong font renders blank. Verify visually by exporting (the Read tool / `pdf.js` can render pages).
- **i18n** — flat key dictionaries (`en` / `th` / `fr`). The Vite OXC transformer rejects curly-quote string delimiters: **use straight quotes** in `.ts` / `.tsx` , especially FR strings.

5. Security & data rules

- **Never commit real employee data.** `seed.cjs` is synthetic (names = job titles, `OFF-001` , DOB `1990-01-01` , "Demo-only" salaries). Real employees come from a **WinDev CSV export** imported via User/Data Management (see [Admin operations](#)) and live in PostgreSQL only — the exported CSV is real PII and stays **out of the repo**.
- **No demo credentials in public docs** — README/docs point to `seed.cjs` for passwords.
- `.env` , `.env.prod` are gitignored. Production fails fast on a weak `JWT_SECRET` (<32 chars), wildcard CORS, or a missing `METRICS_TOKEN` (see `backend/src/config/env.ts`).
- More: [threat-model.md](#).

6. Dev workflow

1. Branch off `main` (don't push straight to `main`).
2. Make the change; keep the surrounding style.
3. Run the gate locally before pushing:

```
npm run verify          # tsc + unit + e2e + build + audit (both workspaces)
```

4. Open a PR. **CI must be green (8 checks):** Lint, Unit tests, Integration (Postgres), E2E smoke, Build, Secret scan, and Build/scan/push images (frontend + backend).
5. Commit messages: conventional prefixes (`feat` , `fix` , `docs` , `chore` , `ci`).

7. Testing

Suite	Command	What it covers
Backend unit	<code>npm test -w backend</code>	scoring, middleware, utils
Backend integration	<code>npm run test:integration</code>	API + Postgres (<code>amw_test</code> DB)
Frontend unit	<code>npm test -w frontend</code>	components (Vitest + jsdom)
Frontend E2E	<code>npm run test:e2e</code>	Playwright (mocked API), desktop + mobile
QA Robot	<code>npm run qa</code>	real UI flows across roles (catches things mocks miss — e.g. the reload-logout bug)

Integration tests need a migrated/seeded `amw_test` DB: `npm run test:integration:setup` first (re-run after schema/seed changes). Full catalog: [system-test-cases.md](#).

8. Common tasks (recipes)

- **Add a field to a user/evaluation:** edit `backend/prisma/schema.prisma` → `npm run db:migrate` → update the service `select`, the Zod schema in the controller, the TS type in `frontend/src/types`, and the form/PDF render.
- **Add a translation:** add the key to all three locales in `translations.ts` (straight quotes), then use `t('key')`.
- **Change scoring weights / thresholds:** weights come from the template/evaluation; attendance thresholds live in `backend/src/services/attendanceService.ts` (and the display table in `frontend/.../AttendanceSection.tsx` — keep them in sync).
- **Add a competency:** `frontend/src/features/evaluations/constants/competency.ts` (set the `positions` array + per-position `descriptions`).
- **Run & screenshot the app:** start servers, then drive with Playwright/headless Chrome (see [reference: screenshot workflow] in the team memory / README screenshots).

9. Admin operations

For admins/HR running the live system:

- **Users** — User Management page (Admin/Developer only): add/edit/delete, reset to a one-time temp password.
- **Bulk employee import (from WinDev)** — employee master data is **pulled from WinDev**: export the employee list from WinDev as a **CSV**, then upload/paste it into the import box on the **User Management / Data Management** page (`employeeImportService` parses it and upserts the records into the database). This is the normal way to load/refresh real employees — keep the exported CSV **outside the repo** (it is real PII; see [Security & data rules](#)).
- **Cycles** — create review periods (supervisory roles); close/delete is admin/developer only.
- **Calibration** — review scores and lock the final 1–5 grade before cycle closure.

- **Reports** — performance BI, department leaderboard, audit trail, CSV export.
- **Password reset** — identity-based (employee number + DOB), **not** email.

Day-2 operations (backups, metrics, incident response): [operations-runbook.md](#).

10. Deployment

Production is Docker Compose (`docker-compose.prod.yml`) with patched, vuln-scanned images.

```
# on the server, with .env.prod filled in (strong secrets):
docker compose -f docker-compose.prod.yml up -d
```

Auto-deploy via GitHub Actions is **gated** behind the repo variable `DEPLOY_ENABLED=true` (set it once `DEPLOY_HOST` / `DEPLOY_USER` / `DEPLOY_SSH_KEY` secrets exist); until then the deploy job skips instead of failing. Full steps: [deploy-checklist.md](#) and [production-readiness.md](#).

Doc map

Doc	Use it for
README	product overview, architecture, screenshots, quick start
api.md	REST API reference
data-model.md	DB schema / relationships
system-test-cases.md	QA/UAT test catalog
deploy-checklist.md	production deploy steps
operations-runbook.md	day-2 ops, backups, incidents
production-readiness.md	go-live gate
threat-model.md	security model
ux-ui-standards.md	design-system conventions